Lógica de Programação | Aula 07

Iteração II: Laços Indeterminados (while)

João Quentino

06 de Março de 2026

Jovem Tech | Turma 04A - Manhã

Recapitulação da Aula 06

O que aprendemos na última aula:

Loop FOR:

- Laços determinados
- Função range()
- Iteração sobre sequências
- Loops aninhados

Padrões:

- Acumulador
- Contador
- Busca
- Validação

Hoje: Loops quando NÃO sabemos quantas vezes repetir!

Agenda de Hoje

Introdução ao WHILE

Exemplos Práticos de WHILE

Controle de Fluxo: break e continue

Loops Infinitos

Padrões Comuns com WHILE

FOR vs WHILE: Quando Usar?

Exercícios Práticos

Projeto Integrador

Boas Práticas com WHILE

Recapitulação

Introdução ao WHILE

FOR vs WHILE: Quando Usar Cada Um?

Escolha baseada no conhecimento prévio:

Use FOR quando:

- Sabe quantas vezes repetir
- Tem uma sequência definida
- Quer iterar sobre lista/string

Exemplos:

- Repetir 10 vezes
- Processar cada aluno
- Desenhar 5 quadrados

Use WHILE quando:

- NÃO sabe quantas vezes
- Depende de uma condição
- Pode nunca executar

Exemplos:

- Até acertar a senha
- Enquanto há saldo
- Até usuário digitar "sair"

O que é um Laço Indeterminado?

O número de repetições depende de uma condição.

Exemplos do cotidiano:

- *Enquanto* estiver com fome, continue comendo
- *Enquanto* tiver dinheiro, continue comprando
- *Enquanto* não acertar, continue tentando

Na programação:

- Validação de entrada (até ser válida)
- Menus interativos (até escolher sair)
- Jogos (até game over)
- Simulações (até condição de parada)

Característica: Pode executar 0, 1, 10 ou infinitas vezes!

Estrutura Básica do WHILE

Sintaxe:

```
while condicao:
    # Codigo executado ENQUANTO condicao for True
    acao1
    acao2
    ...
    # IMPORTANTE: atualizar a condicao!
```

Elementos:

- `while` - palavra-chave
- `condicao` - expressão booleana (como no IF)
- `:` - dois pontos obrigatórios
- **Indentação** - bloco do loop
- **Atualização** - mudar condição eventualmente!

Primeiro Exemplo: Contagem Simples

Contar de 1 a 5:

```python
contador = 1

while contador <= 5:
    print(f"Contagem: {contador}")
    contador += 1  # ESSENCIAL!

print("Loop finalizado!")
```

Saída:

```
Contagem: 1
Contagem: 2
Contagem: 3
Contagem: 4
Contagem: 5
Loop finalizado!
```

Anatomia do WHILE

Três partes essenciais:

```
# 1. INICIALIZACAO (antes do loop)
contador = 1

# 2. CONDICAO (testa antes de cada iteracao)
while contador <= 5:
    print(contador)

    # 3. ATUALIZACAO (dentro do loop)
    contador += 1  # Muda a condicao!
```

Se esquecer a atualização: LOOP INFINITO!

(Veremos isso em detalhes mais adiante)

Exemplos Práticos de WHILE

Exemplo: Validação de Entrada

Pedir número até ser válido (1-10):

```python
numero = int(input("Digite um numero (1-10): "))

while numero < 1 or numero > 10:
    print("Invalido! Tente novamente.")
    numero = int(input("Digite um numero (1-10): "))

print(f"Numero valido: {numero}")
```

Comportamento:

- Se válido na primeira: não entra no loop
- Se inválido: continua pedindo até acertar

Exemplo: Senha com Tentativas Limitadas

Máximo 3 tentativas para acertar a senha:

```python
senha_correta = "python123"
tentativas = 0
max_tentativas = 3
acertou = False

while tentativas < max_tentativas and not acertou:
    senha = input("Digite a senha: ")
    tentativas += 1

    if senha == senha_correta:
        acertou = True
        print("Acesso permitido!")
    else:
        restantes = max_tentativas - tentativas
        print(f"Senha incorreta! Restam {restantes}")
if not acertou:
    print("Conta bloqueada!")
```

Exemplo: Menu Interativo

Menu que repete até escolher sair:

```python
opcao = ""

while opcao != "0":
    print("\n=== MENU ===")
    print("1 - Ver saldo")
    print("2 - Depositar")
    print("3 - Sacar")
    print("0 - Sair")

    opcao = input("Escolha: ")
    if opcao == "1":
        print("Saldo: R$ 1000")
    elif opcao == "2":
        print("Deposito realizado")
    elif opcao == "3":
        print("Saque realizado")
    elif opcao == "0":
        print("Ate logo!")
```

Exemplo: Soma até Zero

Somar números até digitar 0:

```python
soma = 0
numero = int(input("Digite um numero (0 para sair): "))

while numero != 0:
    soma += numero
    numero = int(input("Digite um numero (0 para sair): "))

print(f"Soma total: {soma}")
```

Padrão: "Sentinela- valor especial que encerra o loop

Uso comum: Processar dados até sinal de parada

Controle de Fluxo: break e continue

BREAK: Saindo do Loop

break interrompe o loop imediatamente:

```python
contador = 1

while True:  # Loop "infinito"
    print(f"Iteracao {contador}")

    if contador >= 5:
        break  # Sai do loop!

    contador += 1

print("Loop finalizado!")
```

Quando usar:

- Condição de saída no meio do loop
- Quando encontra o que procura

Exemplo: Busca com BREAK

Procurar um número em uma lista:

```python
numeros = [10, 25, 30, 15, 20, 35, 40]
procurado = 15
indice = 0
encontrado = False

while indice < len(numeros):
    if numeros[indice] == procurado:
        print(f"Encontrado na posicao {indice}!")
        encontrado = True
        break  # Para assim que encontrar

    indice += 1

if not encontrado:
    print("Numero nao encontrado")
```

CONTINUE: Pulando Iteração

continue pula para a próxima iteração:

```
numero = 0

while numero < 10:
    numero += 1

    if numero % 2 == 0:
        continue  # Pula pares

    print(numero)  # Imprime apenas impares
```

Saída: 1, 3, 5, 7, 9

Diferença:

- **break:** sai do loop completamente
- **continue:** pula para próxima iteração

Exemplo: Processamento com CONTINUE

Processar apenas números positivos:

```python
numeros = [5, -3, 10, -7, 15, -2, 20]
indice = 0
soma = 0

while indice < len(numeros):
    numero = numeros[indice]
    indice += 1

    if numero < 0:
        continue  # Ignora negativos

    soma += numero

print(f"Soma dos positivos: {soma}")  # 50
```

BREAK vs CONTINUE: Comparação

Com BREAK:

```
i = 0
while i < 10:
    i += 1
    if i == 5:
        break
    print(i)
# Saida: 1, 2, 3, 4
```

Para quando i=5

Com CONTINUE:

```
i = 0
while i < 10:
    i += 1
    if i == 5:
        continue
    print(i)
# Saida: 1,2,3,4,6,7,8,9,10
```

Pula apenas o 5

Loops Infinitos

O Perigo dos Loops Infinitos

Loop infinito: nunca para de executar!

Consequências:

- Programa trava
- Consome 100% da CPU
- Pode travar o computador
- Precisa forçar encerramento (Ctrl+C)

Causas comuns:

- Esquecer de atualizar a variável de controle
- Condição sempre verdadeira
- Erro lógico na condição
- Atualizar variável errada

Exemplo: Loop Infinito Acidental

CUIDADO! Este código nunca termina:

```python
# ERRO: Esqueceu de incrementar!
contador = 1

while contador <= 5:
    print(f"Contagem: {contador}")
    # FALTOU: contador += 1

# Imprime "Contagem: 1" infinitamente!
```

Problema: `contador` sempre vale 1!

Solução: Sempre atualizar a variável de controle!

Mais Exemplos de Loops Infinitos

Outros erros comuns:

```python
# ERRO 1: Condicao sempre True
while True:
    print("Nunca para!")
    # Faltou break ou condicao de saida

# ERRO 2: Incremento errado
x = 0
while x < 10:
    print(x)
    x -= 1  # ERRO! Diminui em vez de aumentar!

# ERRO 3: Variavel errada
i = 0
j = 5
while i < 10:
    print(i)
    j += 1  # ERRO! Incrementa j, nao i!
```

Como Evitar Loops Infinitos

Checklist de segurança:

1. **Inicialização correta**
   - Variável de controle tem valor inicial adequado?
2. **Condição clara**
   - A condição pode se tornar False?
   - Testei casos extremos?
3. **Atualização presente**
   - Modifico a variável de controle dentro do loop?
   - A modificação aproxima da condição de saída?
4. **Teste mental**
   - Consigo prever quando vai parar?
   - Simulei 2-3 iterações no papel?

Loop Infinito Intencional (com BREAK)

Às vezes queremos loop infinito com saída condicional:

```
while True:  # Loop infinito proposital!
    comando = input("Digite um comando (sair para encerrar): ")

    if comando == "sair":
        break  # Saida intencional

    print(f"Voce digitou: {comando}")
    # Processar comando...

print("Programa encerrado")
```

Padrão válido: Quando a condição de saída está no meio do loop

Importante: Sempre ter um `break` alcançável!

Padrões Comuns com WHILE

Padrão 1: Validação de Entrada

Pedir entrada até ser válida:

```python
idade = -1  # Valor inicial invalido

while idade < 0 or idade > 120:
    try:
        idade = int(input("Digite sua idade (0-120): "))

        if idade < 0 or idade > 120:
            print("Idade invalida!")
    except ValueError:
        print("Digite um numero!")
        idade = -1  # Mantem invalido

print(f"Idade registrada: {idade}")
```

Padrão 2: Menu com Loop

Menu que repete até sair:

```python
continuar = True

while continuar:
    print("\n=== CALCULADORA ===")
    print("1 - Somar")
    print("2 - Subtrair")
    print("3 - Sair")

    opcao = input("Escolha: ")

    if opcao == "1":
        # codigo da soma...
        print("Soma realizada")
    elif opcao == "2":
        # codigo da subtracao...
        print("Subtracao realizada")
    elif opcao == "3":
        continuar = False
```

Padrão 3: Acumulação até Condição

Somar até atingir limite:

```python
soma = 0
numero = 1

while soma < 100:
    soma += numero
    print(f"Adicionado {numero}, soma = {soma}")
    numero += 1

print(f"Parou quando soma = {soma}")
print(f"Ultimo numero adicionado: {numero - 1}")
```

Útil para: Acumular até meta, processar até limite

Padrão 4: Tentativas com Limite

Tentar até acertar ou esgotar tentativas:

```python
import random

numero_secreto = random.randint(1, 10)
tentativas = 0
max_tentativas = 5
acertou = False

while tentativas < max_tentativas and not acertou:
    palpite = int(input(f"Tentativa {tentativas + 1}: "))
    tentativas += 1

    if palpite == numero_secreto:
        acertou = True
        print(f"Parabens! Acertou em {tentativas} tentativas!")
    elif palpite < numero_secreto:
        print("Muito baixo!")
    else:
        print("Muito alto!")
```

FOR vs WHILE: Quando Usar?

Comparação: FOR vs WHILE

Use FOR quando...	**Use WHILE quando...**
Sabe quantas vezes repetir	NÃO sabe quantas vezes
Tem sequência para iterar	Depende de condição
Número fixo de iterações	Número variável
Processar lista/string completa	Parar no meio da sequência
Contagem definida (1 a 10)	Até condição mudar

Regra prática: Se pode usar FOR, use! É mais seguro.

Mesmo Problema, Dois Loops

Somar números de 1 a 10:

Com FOR (preferível):

```python
soma = 0

for i in range(1, 11):
    soma += i

print(soma)  # 55
```

Mais conciso e seguro

Com WHILE (funciona):

```python
soma = 0
i = 1

while i <= 10:
    soma += i
    i += 1

print(soma)  # 55
```

Mais verboso, mais risco

Preferência: FOR quando possível, WHILE quando necessário

Quando WHILE é Obrigatório

Situações onde só WHILE funciona:

```python
# 1. Entrada ate ser valida
senha = ""
while senha != "python":
    senha = input("Digite a senha: ")

# 2. Jogo ate game over
vivo = True
while vivo:
    acao = input("O que fazer? ")
    # logica do jogo...
    if sem_vida:
        vivo = False

# 3. Processar ate fim de arquivo
while arquivo.tem_mais_dados():
    linha = arquivo.ler_proxima()
    # processar linha...
```

Exercícios Práticos

Exercício 1: Adivinhação Simples

Desafio (10 min):

Jogo onde usuário adivinha número de 1 a 10:

```
import random

numero = random.randint(1, 10)
palpite = 0

# Complete o loop while
while _____________:
    palpite = int(input("Adivinhe (1-10): "))

    if palpite < numero:
        print("Muito baixo!")
    elif palpite > numero:
        print("Muito alto!")

print("Parabens! Acertou!")
```

Exercício 2: Média de Notas

Desafio (15 min):

Calcular média de notas (usuário decide quantas):

```
soma = 0
quantidade = 0

nota = float(input("Digite uma nota (-1 para sair): "))

# Complete: soma notas ate digitar -1
while _____________:
    soma += nota
    quantidade += 1
    nota = float(input("Digite uma nota (-1 para sair): "))

if quantidade > 0:
    media = soma / quantidade
    print(f"Media: {media:.2f}")
else:
```

Exercício 3: Fibonacci até Limite

Desafio (15 min):

Gerar Fibonacci até número ultrapassar 1000:

```
a, b = 0, 1
limite = 1000

print(a)
print(b)

# Complete: gere fibonacci ate b > limite
while _______________:
    c = a + b
    print(c)
    a, b = b, c

print(f"Parou em {b}")
```

Exercício 4: Validador de CPF Simulado

Desafio (15 min):

Pedir CPF até ter 11 dígitos numéricos:

```
cpf = ""

while __________________________:
    cpf = input("Digite o CPF (11 digitos): ")

    # Validacoes
    if len(cpf) != 11:
        print("CPF deve ter 11 caracteres!")
    elif not cpf.isdigit():
        print("CPF deve conter apenas numeros!")
    else:
        break  # Valido!

print(f"CPF registrado: {cpf}")
```

Exercício 5: Caixa Eletrônico

Desafio (20 min):

Sistema de caixa eletrônico com menu:

```python
saldo = 1000.0
continuar = True

while continuar:
    print(f"\nSaldo atual: R$ {saldo:.2f}")
    print("1 - Sacar")
    print("2 - Depositar")
    print("3 - Sair")

    # Complete o menu:
    # - Validar valores
    # - Nao permitir saque maior que saldo
    # - Atualizar saldo corretamente
```

Exercício 6: Detector de Palíndromo

Desafio (20 min):

Verificar se palavra é palíndromo usando WHILE:

```python
palavra = input("Digite uma palavra: ").lower()

inicio = 0
fim = len(palavra) - 1
eh_palindromo = True

# Compare caracteres de fora para dentro
while inicio < fim:
    if palavra[inicio] != palavra[fim]:
        eh_palindromo = False
        break

    inicio += 1
    fim -= 1

```

Projeto Integrador

Projeto: Jogo de RPG Simples

Especificações:

1. Jogador tem vida inicial (100 HP)
2. A cada turno, escolhe: atacar, defender ou fugir
3. Inimigo também ataca
4. Batalha continua até:
   - Jogador morrer (HP <= 0)
   - Inimigo morrer (HP <= 0)
   - Jogador fugir
5. Use WHILE para loop principal
6. Use break/continue apropriadamente

Conceitos aplicados:

- WHILE com múltiplas condições
- break para saídas
- random para ataques

Solução Guiada: Parte 1 - Setup

```python
import random

# Estado inicial
hp_jogador = 100
hp_inimigo = 50
turno = 0
fugiu = False

print("=== BATALHA ===")
print(f"Voce: {hp_jogador} HP")
print(f"Inimigo: {hp_inimigo} HP\n")
```

Solução Guiada: Parte 2 - Loop Principal

```
while hp_jogador > 0 and hp_inimigo > 0 and not fugiu:
    turno += 1
    print(f"\n--- Turno {turno} ---")
    print(f"Sua vida: {hp_jogador} HP")
    print(f"Inimigo: {hp_inimigo} HP")

    print("\n1 - Atacar")
    print("2 - Defender")
    print("3 - Fugir")

    acao = input("Escolha: ")

    # (continua no proximo slide...)
```

Boas Práticas com WHILE

Boas Práticas

Diretrizes para loops WHILE seguros:

1. **Sempre inicialize a variável de controle**
2. **Garanta que a condição pode se tornar False**
3. **Atualize a variável dentro do loop**
4. **Evite lógica complexa na condição**
5. **Prefira FOR quando possível**
6. **Use break para saídas claras**
7. **Comente loops não-óbvios**
8. **Teste com valores extremos**

Recapitulação

O que Aprendemos Hoje

Loops WHILE e Controle de Fluxo:

1. **WHILE**: Laços indeterminados
2. **break**: Sair do loop
3. **continue**: Pular iteração
4. **Loops infinitos**: Como evitar
5. **Padrões**: Validação, menus, tentativas
6. **FOR vs WHILE**: Quando usar cada um

Você agora domina TODAS as estruturas de controle básicas!

Próximo módulo: Estruturas de dados!

Desafio para Casa

Projetos para praticar:

1. **Sistema de Login Completo:**
   - 3 tentativas para senha
   - Validação de usuário
   - Menu após login
2. **Jogo da Forca:**
   - Palavra secreta
   - Tentativas limitadas
   - Revelar letras acertadas
3. **Simulador de Investimento:**
   - Simular meses até meta
   - Juros compostos
   - Relatório mensal

Dúvidas e Encerramento

Momento para perguntas!

Contato:

joao.quentino@dcomp.ufs.br

Obrigado!

Próximo módulo: Estruturas de Dados (Listas, Dicionários...)!